현대 AI 개론

PART VI. 생태계와 인프라

오픈소스 모델, 추론 인프라, 코드 어시스턴트

AI를 실제로 운영하는 기술 스택

공냥이

2026

목차

19 오픈소스가 폐쇄형의 80%를 10%의 가격에 제공한다 4

2강 구도가 4강 이상으로 확대됐다 4

오픈 웨이트와 오픈 소스는 다르다 5

오픈소스가 가격의 상한선을 내린다 6

실전에서 어떤 모델을 선택하는가 6

파인튜닝 생태계는 오픈 웨이트 위에서만 작동한다 6

연구 민주화: 검증 가능해야 과학이다 7

20 모델을 만드는 것보다 돌리는 것이 더 비싸다 8

학습 비용은 한 번이고, 서빙 비용은 영원하다 8

KV-Cache: 컨텍스트가 길면 비싼 이유 8

vLLM과 PagedAttention: OS에서 답을 찾다 9

vLLM V1: 2025년의 대규모 업그레이드 10

프로덕션 서빙 아키텍처의 구조 10

GPU 독점은 언제까지 지속되는가 11

컨텍스트 엔지니어링이 곧 비용 엔지니어링이다 11

21 코드 어시스턴트와 코드 에이전트는 근본적으로 다르다 13

보조하는 도구와 일하는 도구의 차이 13

세 제품이 점유하는 영역은 서로 다르다 14

벤치마크가 말해주는 것과 말해주지 않는 것 14

작업별 최적 선택은 이렇게 갈라진다 15

컨텍스트 인식의 깊이가 도구의 품질을 결정한다 15

Windsurf와 후발 주자들 16

같이 쓰는 것이 표준이 됐다 16

22 5개 계층이 하나의 생태계에서 만난다 18

안전이 제품 경쟁력이 되는 회사 18

5개 계층이 Claude에서 어떻게 매핑되는가 18
Claude Code가 코딩 도구에서 범용 에이전트로 진화한 이유 19
Claude 모델 계보: 2026년 4월 현재 20
MCP와 Skills가 만나는 지점 20
Claude를 제대로 쓰는 다섯 가지 원칙 21
생태계 수준에서 본 Claude의 위치 21

CHAPTER 19

오픈소스가 폐쇄형의 80%를 10%의 가격에 제공한다

2024년까지 오픈소스 LLM은 폐쇄형 모델의 열등한 대안이었다. 2026년 현재 그 격차는 벤치마크 기준 5 10% 이내로 줄었고, 비용은 10분의 1 이하다. 이 역전이 생태계 전체를 바꾸고 있다.

2강 구도가 4강 이상으로 확대됐다

2024년까지 오픈소스 LLM의 양대 축은 Meta의 Llama와 Mistral이었다. 2026년 4월 현재, 지형도가 근본적으로 달라졌다. 4개 이상의 진영이 벤치마크 상위권을 다투고 있다.

모델	개발사	파라미터	특징
Llama 4 Maverick	Meta	400B (MoE, 17B 활성)	MMLU 85.5%, 오픈 모델 최고 성능
Llama 4 Scout	Meta	109B (MoE, 17B 활성)	10M 토큰 컨텍스트
Qwen 3.5	Alibaba	MoE (경량 활성)	코딩, 수학 벤치마크 선두
Mistral Large 3	Mistral	123B	80개 이상 언어 지원, Apache 2.0
DeepSeek V3.2	DeepSeek	MoE	GPT-5급 추론, MIT 라이선스
DeepSeek R1	DeepSeek	MoE	o1급 추론 모델, 입력 토큰 27배 저렴

이 표에서 세 가지가 눈에 띈다. 첫째, MoE(Mixture of Experts, 전문가 혼합)가 표준이 됐다. Llama 4 Maverick은 400B 파라미터를 갖고 있지만 실제 활성화되는 것은 17B에 불과하다. 400B 전체를 매번 돌리는 것이 아니라, 입력에 따라 필요한 전문가 모듈만 선택적으로 활성화하는 구조다. PART I에서 다룬 Chinchilla 스케일링 법칙의 실전 적용인 셈이다. 전체를 키우는 대신 필요한 부분만 활성화한다.

둘째, 중국 진영이 따라잡았다. Alibaba의 Qwen 3.5가 2026년 3월 기준으로 코딩, 수학, 지시 따르기, 롱컨텍스트 추론에서 여러 벤치마크를 선도하고 있다. DeepSeek V3.2는 GPT-5급 추론 태스크에서 경쟁한다.

셋째, 라이선스가 열렸다. Mistral은 이전의 제한적 라이선스에서 Apache 2.0으로 전환했다. DeepSeek은 MIT 라이선스를 채택했다. 상업적 사용에 사실상 제한이 없어진 것이다.

오픈 웨이트와 오픈 소스는 다르다

용어를 정확히 써야 한다. 대부분의 **오픈소스 LLM**은 엄밀히 말하면 오픈 웨이트(Open-Weight)다.

공개 범위	오픈 소스	오픈 웨이트
가중치	공개	공개
학습 코드	공개	비공개 (대부분)
학습 데이터	공개	비공개 (대부분)
대표 사례	—	Llama, Mistral, Qwen

가중치를 공개한다는 것은 모델을 다운로드해서 직접 돌릴 수 있다는 뜻이다. 그러나 그 모델이 어떤 코드로, 어떤 데이터로 학습됐는지는 알 수 없다. 자동차를 사서 운전할 수는 있지만, 설계도와 부품 목록은 받지 못하는 것과 같다.

이 구분이 왜 중요한가. 학습 코드와 데이터가 공개되지 않으면 결과를 재현할 수 없다. 과학의 기본 원칙인 재현성이 보장되지 않는 것이다. 이 점에서 DeepSeek이 주목받는 이유가 있다. DeepSeek은 학습 방법론까지 상세히 공개한다. R1-Zero가 순수 강화학습(Reinforcement Learning, RL)만으로 추론 능력의 출현을 보여준 것은, 방법론이 공개됐기 때문에 전 세계 연구자들이 검증할 수 있었다. 폐쇄형 모델에서만 이 발견이 나왔다면 검증 자체가 불가능했을 것이다.

오픈소스 LLM — 누가 이기고 있는가

2026년 주요 오픈 웨이트 모델

모델	개발사	파라미터	특징	라이선스
Llama 4 Maverick	Meta	400B MoE	MMLU 85.5%, 오픈 최고	Llama
Llama 4 Scout	Meta	109B MoE	10M 컨텍스트	Llama
Qwen 3.5	Alibaba	MoE (경량)	코딩/수학 강세	Apache 2.0
Mistral Large 3	Mistral	123B	80+ 언어	Apache 2.0
DeepSeek V3.2	DeepSeek	MoE	GPT-5급 추론	MIT
DeepSeek R1	DeepSeek	MoE	o1급, 27배 저렴	MIT

그림 1: 2026년 LLM 지형도. 폐쇄형과 오픈 웨이트 모델의 성능 격차가 급격히 줄어든 현황을 보여준다.

이 그림에서 주목할 것은 상위권의 밀집도다. 2024년에는 폐쇄형과 오픈 웨이트 사이에 뚜렷한 격차가 있었지만, 2026년에는 벤치마크 상위 10개 모델 중 절반 이상이 오픈 웨이트다.

오픈소스가 가격의 상한선을 내린다

DeepSeek R1이 o1 대비 입력 토큰 27배 저렴하게 출시됐다. 이 가격 차이가 시장에 미친 영향은 직접적이었다. OpenAI가 o3-mini를 당초 일정보다 앞당겨 출시한 것이 대표적 사례다.

오픈소스의 경제적 기능은 가격의 상한선을 내리는 것이다. 폐쇄형 모델이 아무리 우수해도, 오픈소스가 80% 성능을 10% 가격에 제공하면 시장이 움직인다. 기업 입장에서 20%의 성능 차이를 위해 10배의 비용을 지불할 이유가 있는지 묻게 되기 때문이다.

이 구조는 소프트웨어 산업에서 반복적으로 나타난 패턴이다. Linux가 상용 Unix의 가격을 끌어내렸고, MySQL이 Oracle의 독점을 약화시켰으며, Android가 모바일 OS 시장을 재편했다. 오픈소스 LLM도 같은 역할을 하고 있다.

> **오픈소스 LLM의 가치는 폐쇄형을 대체하는 것이 아니라, 폐쇄형이 과도한 가격을 매기지 못하게 하는 데 있다. 경쟁이 가격을 내리고, 가격이 내려가면 접근성이 올라간다.**

실전에서 어떤 모델을 선택하는가

모델 선택은 단일 벤치마크로 결정할 문제가 아니다. 용도에 따라 최적의 선택이 달라진다.

용도	추천 모델	이유
범용 (시작점)	Llama 4 70B	가장 범용적이며 배포 지원이 풍부하다
추론 태스크	DeepSeek R1/V3.2	GPT-5급 추론 성능에 MIT 라이선스
다국어	Mistral Large 3	80개 이상 언어를 지원하며 Apache 2.0
로컬 실행	Qwen 3.5 (10B 활성)	MacBook에서 실행 가능한 수준
롱컨텍스트	Llama 4 Scout	10M 토큰 컨텍스트 지원
코딩/수학	Qwen 3.5	해당 벤치마크에서 선두

이 표에서 주목할 것은 **하나의 모델이 모든 용도에서 최고가 아니라는 점**이다. 범용에서는 Llama가 강하지만, 추론에서는 DeepSeek이, 다국어에서는 Mistral이, 코딩에서는 Qwen이 앞선다. 2024년에는 GPT-4 하나면 충분하다는 인식이 있었다. 2026년에는 용도에 따라 선택하는 것이 표준이 됐다.

파인튜닝 생태계는 오픈 웨이트 위에서만 작동한다

오픈 웨이트가 중요한 또 다른 이유는 파인튜닝(fine-tuning) 때문이다. 도메인 특화 모델, 예컨대 의료, 법률, 금융 분야의 전문 모델은 범용 모델을 해당 도메인의 데이터로 추가 학습시켜서 만든다. 이 추가 학습에는 가중치에 대한 접근이 필수다.

폐쇄형 모델에서는 이것이 불가능하거나, API를 통한 제한적 파인튜닝만 가능하다. 가중치를 직접 다룰 수 없으므로 학습 과정을 세밀하게 통제할 수 없다. 오픈 웨이트가 있어야 LoRA(Low-Rank Adaptation, 저랭크 적응)나 QLoRA 같은 효율적 파인튜닝 기법을 적용할 수 있다. 이 기법들은 전체 가중치를 수정하는 대신 소수의 파라미터만 조정해서, GPU 1 2장으로도 파인튜닝을 가능하게 한다.

2026년 현재, 도메인 특화 모델의 대부분이 Llama, Qwen, Mistral 기반이다. 이 세 모델이 가중치를 공개하고 있기 때문이다. 오픈 웨이트가 없었다면 도메인 특화 AI의 발전 속도는 훨씬 느렸을 것이다.

연구 민주화: 검증 가능해야 과학이다

R1-Zero의 사례는 오픈소스가 연구에 미치는 영향을 보여준다. R1-Zero는 순수 강화학습만으로 추론 능력이 출현한다는 것을 보여줬다. 지도학습 데이터 없이, 보상 신호만으로 모델이 단계적 추론을 학습하는 현상이다.

이 발견이 의미 있는 이유는 검증 가능하기 때문이다. DeepSeek이 학습 방법론을 상세히 공개했으므로, 전 세계 연구자들이 동일한 실험을 재현하고 결과를 확인할 수 있다. 재현할 수 없는 결과는 주장이지 과학이 아니다.

반대 사례도 있다. OpenAI의 o1이 보여준 추론 능력의 구체적 학습 방법은 공개되지 않았다. 결과만 볼 수 있고 과정은 블랙박스다. 성능은 인상적이지만, 왜 작동하는지를 외부에서 분석할 수 없다. 의학에서 임상시험 데이터를 공개하지 않는 신약을 신뢰하기 어려운 것과 같은 원리다.

기준	오픈 (DeepSeek R1)	폐쇄 (OpenAI o1)
성능	o1급	최고 수준
방법론 공개	상세 공개	비공개
재현 가능	가능	불가능
비용	입력 토큰 27배 저렴	기준
라이선스	MIT	API만 제공

이 표의 핵심은 성능이 비슷한데 나머지 모든 항목에서 차이가 난다는 점이다. 성능이 동등해지는 순간, 공개 여부와 비용이 선택의 결정 요인이 된다.

> **오픈소스 LLM은 세 가지 역할을 동시에 수행한다. 가격의 상한선을 내리고, 도메인 특화 모델의 기반이 되며, 연구의 재현성을 보장한다. 이 세 역할이 AI 생태계 전체의 건강을 지탱한다.**

다음 글에서는 이 모델들을 실제로 서비스에 올리는 추론 인프라를 다룬다. 모델의 성능만큼이나 그 모델을 효율적으로 돌리는 기술이 중요하기 때문이다.

Source: Meta — Llama 4 Scout/Maverick 2025-04; Alibaba — Qwen 3.5 2026-03; Mistral — Large 3 Apache 2.0; DeepSeek — V3.2, R1/R1-Zero MIT; till-freitag.com 2026; Hoffmann et al. (Chinchilla) 2022

CHAPTER 20

모델을 만드는 것보다 돌리는 것이 더 비싸다

AI 비용의 대부분은 학습이 아니라 서빙에서 발생한다. 학습은 한 번이지만 서빙은 매 요청마다 반복된다. vLLM과 PagedAttention이 이 비용 구조를 근본적으로 바꿨다.

학습 비용은 한 번이고, 서빙 비용은 영원하다

많은 사람이 AI 비용을 학습 비용과 동일시한다. GPT-4의 학습에 1억 달러 이상이 들었다는 보도가 이 인식을 강화했다. 그러나 이 인식은 절반만 맞다.

학습은 한 번이다. 모델을 만들 때 한 번 돈을 쓴다. 서빙, 즉 추론(inference)은 매 요청마다 돈이 든다. 사용자가 100만 명이고 하루 평균 10번씩 쓰면 하루 1,000만 건의 추론이 발생한다. 1년이면 36억 건이다.

식당을 비유로 쓰자. 레시피를 개발하는 비용은 한 번이다. 그러나 매일 수천 명의 손님에게 요리를 내놓는 비용은 매일 반복된다. 장기적으로 보면 레시피 개발비보다 매일의 운영비가 압도적으로 크다. AI 서빙도 같은 구조다. 모델이 대중화될수록 학습 비용 대비 서빙 비용의 비율이 기하급수적으로 커진다.

이 때문에 추론 인프라(inference infrastructure)가 AI 산업의 핵심 경쟁 영역이 됐다. 같은 모델이라도 어떤 인프라 위에서 돌리느냐에 따라 비용이 2 4배 달라진다.

KV-Cache: 컨텍스트가 길면 비싼 이유

트랜스포머 모델이 토큰을 생성하는 과정을 이해해야 비용 구조가 보인다. 트랜스포머는 새 토큰을 만들 때마다 이전에 생성된 모든 토큰을 참조한다. 구체적으로는 각 토큰의 Key 벡터와 Value 벡터를 참조한다. Key는 **내가 어떤 정보를 갖고 있는가**를 나타내는 벡터이고, Value는 **내가 제공할 수 있는 구체적 정보**를 담은 벡터다. 이 두 벡터를 합쳐서 KV(Key-Value)라고 부른다.

KV-Cache(KV 캐시)는 이 KV 벡터들을 메모리에 저장해두는 것이다. 새 토큰을 만들 때마다 이전 토큰의 KV를 처음부터 다시 계산하면 극도로 비효율적이다. 한 번 계산한 KV를 캐시에 저장해두면 재

계산 없이 바로 참조할 수 있다. 수학 시험에서 중간 계산 결과를 여백에 적어두는 것과 같다. 매번 처음부터 다시 푸는 대신 적어둔 결과를 참조한다.

문제는 이 캐시가 차지하는 메모리가 컨텍스트 길이에 비례해서 커진다는 점이다. 그리고 기존 시스템은 이 메모리의 60 80%를 낭비하고 있었다.

요청	메모리 할당과 낭비
요청 A: 컨텍스트 1,000토큰	4,000토큰분 할당 → 3,000토큰분 낭비
요청 B: 컨텍스트 3,000토큰	4,000토큰분 할당 → 1,000토큰분 낭비

왜 이런 낭비가 발생하는가. 요청이 최종적으로 몇 토큰을 생성할지 미리 알 수 없기 때문이다. 시스템은 최대치를 기준으로 메모리를 미리 할당한다. 대부분의 요청은 최대치를 쓰지 않는다. 남는 공간은 비어 있지만 다른 요청이 쓸 수 없다. 60 80% 낭비의 원인이 여기에 있다.

vLLM과 PagedAttention: OS에서 답을 찾다

2023년, UC Berkeley의 Woosuk Kwon 팀이 이 문제를 해결하는 시스템을 발표했다. vLLM이라는 이름의 이 시스템은 SOSP(Symposium on Operating Systems Principles, 운영체제 원리 심포지엄) 2023에서 발표됐다. 운영체제 학회에서 AI 시스템이 발표된 것이 의미심장하다. 해법이 OS에서 왔기 때문이다.

아이디어는 이렇다. 운영체제는 프로그램의 메모리를 페이지(page)라는 고정 크기 블록으로 나눠서 관리한다. 프로그램이 연속된 메모리를 요구하지만, OS는 물리적으로는 비연속적인 페이지들을 이어 붙여서 연속인 것처럼 보이게 한다. 이것이 가상 메모리(virtual memory)다.

vLLM은 이 가상 메모리 기법을 KV-Cache에 적용했다. PagedAttention(페이지드 어텐션)이라는 이름의 이 기법은 KV-Cache를 고정 크기 페이지로 나눠서 관리한다. 연속된 큰 블록을 미리 할당하는 대신, 필요할 때마다 페이지 단위로 할당한다.

지표	기존 시스템	vLLM (PagedAttention)
KV-Cache 메모리 낭비	60 80%	4% 미만
처리량 (동일 지연 시간)	기준	2 4배
GPU 활용률 (고동시성)	40 60%	85 92%

이 숫자의 의미를 풀어보자. 메모리 낭비가 60 80%에서 4% 미만으로 줄었다는 것은, 같은 GPU로 2배 이상의 요청을 동시에 처리할 수 있다는 뜻이다. 처리량이 2 4배 늘었다는 것은, 같은 하드웨어 비용으로 2 4배 많은 사용자를 서비스할 수 있다는 뜻이다. GPU가 가장 비싼 자원인 AI 서빙에서, 같은 GPU로 2 4배 효율을 내는 것은 비용을 절반 이하로 줄이는 것과 같다.

> **vLLM의 PagedAttention은 AI 추론의 비용 구조를 바꿨다. 해법은 새로운 AI 알고리즘이 아니라 1960년대부터 있던 OS의 가상 메모리 기법이었다.**

vLLM V1: 2025년의 대규모 업그레이드

2025년 1월, vLLM V1이 출시됐다. 이전 버전 대비 1.7배 속도가 향상됐다. 세 가지 핵심 개선이 있었다.

첫째, 제로 오버헤드 프리픽스 캐싱(zero-overhead prefix caching)이다. 프리픽스 캐싱이란 여러 요청이 같은 시스템 프롬프트로 시작할 때, 그 공통 부분의 KV-Cache를 재사용하는 기법이다. PART III에서 **안정적인 것을 앞에, 변하는 것을 뒤에 배치하라**고 했던 것이 여기서 비용 절감으로 직접 이어진다. 시스템 프롬프트가 10K 토큰이고 모든 요청에 공통이라면, 프리픽스 캐싱은 매 요청마다 10K 토큰분의 연산을 절약한다.

둘째, 추측 디코딩(Speculative Decoding)이다. 작은 모델이 답변의 초안을 빠르게 생성하고, 큰 모델이 그 초안을 검증한다. 초안의 대부분이 채택되면 전체 속도가 올라간다. 시험 답안을 빠르게 초안 작성하고, 꼼꼼하게 검토하는 것과 같다.

셋째, 분리형 프리필/디코드(Disaggregated Prefill/Decode)다. 입력을 처리하는 단계(프리필)와 토큰을 생성하는 단계(디코드)를 물리적으로 분리한다. 두 단계의 연산 특성이 다르기 때문이다. 프리필은 대량의 토큰을 한꺼번에 처리하므로 연산 집약적이고, 디코드는 토큰을 하나씩 생성하므로 메모리 집약적이다. 둘을 분리하면 각각에 최적화된 하드웨어를 배정할 수 있다.

2026년 현재 vLLM은 LLM 추론 서빙의 사실상 표준이다.

그림 2: KV-Cache의 메모리 구조. PagedAttention이 메모리 낭비를 60 80%에서 4% 미만으로 줄인 원리를 보여준다.

이 그림에서 주목할 것은 기존 방식과 PagedAttention 방식의 메모리 할당 패턴 차이다. 기존 방식은 큰 연속 블록을 할당하고 대부분을 비워두지만, PagedAttention은 필요한 페이지만 할당한다.

프로덕션 서빙 아키텍처의 구조

가장 단순한 서빙 구성은 사용자 요청이 API 서버를 거쳐 vLLM이 돌아가는 GPU 서버에 도달하고, 응답이 돌아오는 구조다. 프로덕션 환경에서는 이것이 복잡해진다.

구성 요소	역할
로드밸런서	요청을 여러 API 서버에 분산

API 서버 (복수)	인증, 요율 제한, 요청 전처리
vLLM 클러스터 (GPU 복수)	모델 추론 실행, KV-Cache 관리
프리픽스 캐시	공통 시스템 프롬프트의 KV-Cache 재사용

이 구조에서 프리픽스 캐시가 비용에 큰 영향을 준다. 같은 시스템 프롬프트로 시작하는 요청이 반복되면 해당 부분의 KV-Cache를 재사용한다. PART III에서 다뤘던 컨텍스트 배치 원칙이 여기서 실질적 비용 차이를 만드는 셈이다.

GPU 독점은 언제까지 지속되는가

2026년 현재 AI 추론의 대부분은 NVIDIA GPU에서 돌아간다. H100, H200이 프로덕션 환경의 표준이다. 그러나 대안이 등장하고 있다.

하드웨어	개발사	특성
LPU	Groq	추론 전용 칩. 지연 시간이 GPU 대비 수배 수십배 빠르다
TPU v6	Google	학습과 추론 모두 지원. Google Cloud 전용
Trainium/Inferentia	AWS	AWS 전용 칩. 자체 클라우드 내 최적화

왜 전용 칩이 나오는가. GPU는 원래 그래픽 처리를 위해 설계된 칩이다. AI 추론에 효율적으로 작동하지만, AI 추론만을 위해 최적화된 것은 아니다. 전용 칩은 불필요한 회로를 제거하고 AI 추론에 필요한 연산만 남겨서 효율을 극대화한다. 범용 트럭과 전용 배달차의 차이와 같다. 범용 트럭은 무엇이든 실을 수 있지만, 배달만 하려면 전용 배달차가 더 효율적이다.

그러나 2026년 현재 NVIDIA의 생태계가 압도적으로 강하다. CUDA라는 프로그래밍 환경, 수만 개의 최적화된 라이브러리, 10년 이상 축적된 개발자 커뮤니티가 진입 장벽을 만들고 있다. 하드웨어 성능만으로는 생태계를 이기기 어렵다.

컨텍스트 엔지니어링이 곧 비용 엔지니어링이다

이 장의 내용이 왜 모델을 서빙하지 않는 사람에게도 중요한가. 이유는 단순하다. 컨텍스트 길이가 API 가격에 직접 반영되기 때문이다.

컨텍스트 10K 토큰으로 질문하는 것과 100K 토큰으로 질문하는 것의 비용 차이는 10배가 아니다. KV-Cache 비용이 비선형으로 증가하므로 10배 이상이 될 수 있다. 같은 질문에 같은 답을 얻으면서 컨텍스트를 절반으로 줄일 수 있다면, 비용도 절반 이하로 줄어든다.

PART III에서 다뤘던 컨텍스트 엔지니어링의 원칙들, 필요한 정보만 넣고 불필요한 것은 압축하고 안정적인 것을 앞에 배치하는 것, 이 모든 것이 성능만이 아니라 비용도 최적화한다. 컨텍스트 엔지니어링은 성능 엔지니어링인 동시에 비용 엔지니어링이다.

> **추론 인프라를 이해하면 비용을 통제할 수 있다. vLLM의 PagedAttention이 서빙 비용을 2 4배 줄였듯이, 컨텍스트 엔지니어링은 사용자 측에서 비용을 줄이는 방법이다. 모델 성능과 비용 효율은 분리할 수 없다.**

다음 글에서는 이 인프라 위에서 실제로 동작하는 코드 어시스턴트를 비교한다. Copilot, Cursor, Claude Code가 각각 어떤 문제를 풀고, 어디서 갈라지는지를 살펴본다.

Source: Kwon et al. (SOSP) 2023 — vLLM, PagedAttention; vLLM V1 2025-01; NVIDIA H100/H200; Groq LPU; Google TPU v6

CHAPTER 21

코드 어시스턴트와 코드 에이전트는 근본적으로 다르다

Copilot, Cursor, Claude Code 중 뭘 써야 하느냐는 2026년 개발자가 가장 많이 묻는 질문이다. 답은 하나가 아니다. 이 셋은 같은 범주가 아니기 때문이다.

보조하는 도구와 일하는 도구의 차이

자동차와 비행기 중 뭐가 좋으냐고 묻는 사람에게는 **어디를 가느냐**고 되물어야 한다. 동네 마트에는 자동차가, 서울에서 부산까지는 비행기가 맞다. 코드 어시스턴트와 코드 에이전트의 관계가 이와 같다.

코드 어시스턴트는 자동완성(autocomplete) 중심이다. 개발자가 코드를 작성하는 도중에 AI가 다음 줄을 제안한다. 개발자가 Tab을 눌러 수락하거나 무시한다. 주도권은 개발자에게 있다. AI는 보조한다. GitHub Copilot이 여기에 해당하고, Cursor의 인라인 자동완성도 이 범주다.

코드 에이전트는 자율 실행 중심이다. 개발자가 목표를 준다. **이 버그를 고쳐줘.** AI가 스스로 파일을 찾고, 수정하고, 테스트하고, 완료를 보고한다. 주도권은 AI에게 있다. 개발자는 감독한다. Claude Code가 여기에 해당한다.

기준	코드 어시스턴트	코드 에이전트
작동 방식	다음 줄 제안 (자동완성)	목표를 받아 자율 실행
주도권	개발자	AI
개발자 역할	작성자	감독자
대표 제품	Copilot, Cursor 인라인	Claude Code, Cursor Agent

이 구분이 중요한 이유는 비교의 기준이 달라지기 때문이다. 어시스턴트는 **얼마나 정확하게 제안하느냐**로 평가한다. 에이전트는 **목표를 얼마나 자율적으로 달성하느냐**로 평가한다. 두 범주를 같은 기준으로 비교하면 엉뚱한 결론이 나온다.

세 제품이 점유하는 영역은 서로 다르다

각 제품의 핵심 특성을 구체적으로 살펴보자.

GitHub Copilot은 인라인 자동완성의 대명사다. VS Code와 JetBrains에 가장 깊이 통합되어 있으며 사용자 수가 가장 많다. 기존 IDE 워크플로우를 유지하면서 보조를 받고 싶을 때 적합하다. 에이전트 모드(Copilot Workspace)를 추가하고 있지만 아직 핵심은 자동완성이다.

Cursor는 AI 네이티브 IDE다. 에디터 자체가 AI 중심으로 설계됐다. 가장 큰 차별점은 코드 인덱싱(code indexing)이다. 프로젝트 전체를 인덱싱해서, 파일 하나가 아니라 프로젝트 전체를 이해한 상태에서 제안한다. **이 프로젝트에서 인증은 어떻게 작동하는가**라고 물으면 프로젝트 전체를 참조해서 답한다. Agent 모드도 있어서 멀티파일 수정이 가능하다.

Claude Code는 터미널 기반 에이전트 하네스다. PART III에서 다뤘던 하네스의 대표적 구현이다. 자율성이 가장 높다. 파일 탐색, 수정, 테스트, 커밋까지 자동으로 수행한다. MCP(Model Context Protocol)로 GitHub, Slack, 데이터베이스를 연결하고, Skills로 프로젝트 규칙을 관리한다. 코딩 도구가 아니라 범용 에이전트 인프라에 가깝다.

코드 어시스턴트 비교 2026

용도에 따라 선택이 다르다

	Claude Code	Cursor	Copilot	Windsurf
핵심	자율 에이전트	AI 네이티브 IDE	인라인 자동완성	플로우 기반
SWE-bench	72-80%	~52%	~56%	—
컨텍스트	1M	모델 의존	모델 의존	모델 의존
강점	멀티파일 자율	코드 인덱싱	IDE 통합	가벼움
가격/월	0-200	0	0	무료~5

그림 3: 코드 어시스턴트와 에이전트의 스펙트럼. 자동완성에서 자율 실행까지 각 제품이 점유하는 영역을 보여준다.

이 그림에서 주목할 것은 각 제품이 스펙트럼 위에서 서로 다른 위치를 점유하고 있다는 점이다. 왼쪽의 자동완성 영역에는 Copilot이, 중간에는 Cursor가, 오른쪽의 자율 실행 영역에는 Claude Code가 있다.

벤치마크가 말해주는 것과 말해주지 않는 것

숫자로 비교해보자.

지표	Claude Code	Cursor	Copilot
SWE-bench Verified	72 80%	약 52%	약 56%
컨텍스트 윈도우	1M 토큰	모델 의존	모델 의존
자동완성 수락률	— (에이전트 중심)	72%	—

월 가격	20 200달러	20달러	10달러

SWE-bench Verified에서 Claude Code가 72 80%로 압도적이다. 그러나 이 벤치마크는 **자율적으로 버그를 고치는** 능력을 측정한다. 일상적 코딩에서의 생산성과는 다른 지표다.

Cursor의 자동완성 수락률 72%는 **타이핑 중 AI 제안을 얼마나 수락하는가**를 측정한다. Claude Code는 자동완성을 하지 않으므로 이 지표가 존재하지 않는다. 비행기의 항속거리와 자동차의 연비를 비교하는 것과 같다. 측정하는 것 자체가 다르다.

2026년 개발자 서베이에 따르면 개발자의 73%가 AI 코딩 도구를 사용하며, 경험 있는 개발자의 평균 도구 수는 2.3개다. 하나만 쓰는 것이 아니라 작업별로 선택한다는 뜻이다.

> **벤치마크만 보고 선택하면 안 된다. 어떤 작업을 하느냐가 선택 기준이다. SWE-bench가 높다고 일상 코딩에서 최고라는 뜻이 아니다.**

작업별 최적 선택은 이렇게 갈라진다

실전 경험에 기반한 시나리오별 선택 기준이다.

시나리오	최적 도구	이유
간단한 함수 작성	Copilot 또는 Cursor 인라인	Tab 한 번이면 끝난다
파일 단위 리팩토링	Cursor Agent	파일 컨텍스트를 이미 이해하고 있다
100개 파일 마이그레이션	Claude Code	서브에이전트 병렬화와 CLAUDE.md 규칙 관리가 필요하다
프로젝트 전체 이해 기반 질문	Cursor	코드 인덱싱이 강점이다
외부 서비스 연결 작업	Claude Code	MCP로 GitHub, Slack, DB 연결이 가장 깊다

이 표의 핵심은 **하나로 모든 것을 해결하려 하지 말라**는 것이다. 간단한 함수를 작성할 때 Claude Code를 꺼내는 것은 동네 마트에 비행기를 타고 가는 것이다. 100개 파일에 걸친 마이그레이션을 Copilot의 자동완성으로 하려는 것은 대양을 자동차로 건너려는 것이다.

컨텍스트 인식의 깊이가 도구의 품질을 결정한다

코드 어시스턴트의 품질을 결정하는 핵심 요소는 컨텍스트 인식의 깊이다. 같은 줄의 코드를 제안하더라도, 현재 파일만 보고 제안하는 것과 프로젝트 전체를 이해하고 제안하는 것은 결과가 다르다.

Copilot의 초기 버전은 현재 파일과 열린 탭 정도만 참조했다. 함수 시그니처를 보고 다음 줄을 예측하는 수준이었다. Cursor는 프로젝트 전체를 인덱싱해서 파일 간의 의존성을 파악한다. 인증 로직이 별도 모듈에 있어도, 현재 파일에서 인증 관련 코드를 작성하면 그 모듈의 API를 참조해서 제안한다.

Claude Code는 한 단계 더 나아간다. CLAUDE.md에 정의된 프로젝트 규칙, MCP로 연결된 외부 서비스의 스키마, 이전 세션의 맥락까지 컨텍스트에 포함된다. 코드뿐 아니라 프로젝트의 규칙과 관행까지 이해한 상태에서 작업하는 것이다.

PART III에서 컨텍스트 엔지니어링의 핵심이 **다음 스텝에 딱 맞는 정보를 넣는 것**이라고 했다. 코드 어시스턴트에서도 같은 원리가 적용된다. 더 정확한 컨텍스트를 더 많이 참조할수록 제안의 품질이 올라간다. 이 차이가 벤치마크 숫자보다 실제 생산성에 더 큰 영향을 미친다.

Windsurf와 후발 주자들

Windsurf(이전 Codeium)는 **플로우(flow)** 기반 접근을 택했다. AI와의 상호작용을 자연스러운 흐름으로 만드는 것이 목표다. 무료 티어가 관대해서 비용에 민감한 개발자에게 적합하다. 세 대 제품 대비 기능은 제한적이지만 가벼움이 장점이다.

2026년에는 이 외에도 수십 개의 코드 어시스턴트가 등장했다. JetBrains가 자체 AI 어시스턴트를 강화하고, Amazon의 CodeWhisperer가 Q Developer로 리브랜딩됐으며, Google의 Gemini Code Assist도 경쟁에 뛰어들었다. 시장이 성숙해지면서 자동완성은 상품화(commoditization)되고 있다. 차별화는 에이전트 영역에서 벌어지고 있다.

같이 쓰는 것이 표준이 됐다

2026년에 가장 흔한 조합은 이렇다. 일상적 코딩에서는 Cursor로 인라인 자동완성과 파일 단위 에이전트를 쓰고, 큰 작업에서는 Claude Code로 멀티파일 에이전트와 MCP, Skills를 활용한다.

이 조합이 자연스러운 이유가 있다. 코딩 작업의 80%는 작은 수정이다. 변수 이름 변경, 함수 추가, 에러 핸들링 보완 같은 것들이다. 이런 작업에는 자동완성이 가장 효율적이다. 나머지 20%는 큰 작업이다. 아키텍처 변경, 대규모 마이그레이션, 새 기능 구현 같은 것들이다. 이런 작업에는 에이전트가 필요하다.

이메일은 Gmail, 문서는 Notion을 쓰듯이, 코딩 도구도 용도별로 선택하는 것이 2026년의 표준이다. 하나의 도구에 모든 것을 맡기려는 시도는 효율적이지 않다.

작업 유형 (비율)	도구	방식
소규모 수정 (80%)	Cursor	인라인 자동완성 + 파일 단위 에이전트
대규모 작업 (20%)	Claude Code	멀티파일 에이전트 + MCP + Skills

> **코드 어시스턴트의 선택은 도구의 우열이 아니라 작업의 성격에 달려 있다. 2026년 개발자는 평균 2.3개의 AI 코딩 도구를 사용하며, 작업별로 최적의 도구를 선택한다.**

다음 글에서는 이 도구들 중 하나인 Claude Code가 속한 Claude 생태계 전체를 조망한다. 개별 제품이 아니라 생태계 수준에서 어떤 구조가 만들어지고 있는지를 살펴본다.

Source: GitHub Copilot; Cursor; Claude Code (Anthropic); SWE-bench 2026; 2026 AI Coding Tool 비교 리포트; The Pragmatic Engineer 2025-09-23

CHAPTER 22

5개 계층이 하나의 생태계에서 만난다

이 책의 PART III에서 다뤘던 프롬프트, 컨텍스트 엔지니어링, 하네스, MCP, Skills의 5개 계층은 추상적 개념이 아니다. Claude 생태계에서 이 다섯 계층이 구체적 제품으로 구현되어 있다.

안전이 제품 경쟁력이 되는 회사

Anthropic이 다른 AI 회사와 다른 지점은 하나다. 대부분의 AI 회사가 **먼저 만들고 나중에 안전**이라면, Anthropic은 **안전이 곧 제품 경쟁력**이라는 입장을 취한다.

이것이 왜 중요한지를 앞 글들과 연결하면 명확해진다. 환각 감소, 가드레일 품질, 프롬프트 인젝션 방어, 이 모든 것이 안전 연구에서 나왔다. 헌법 AI(Constitutional AI)는 인간 피드백 없이도 AI가 스스로 행동을 교정하게 하는 기법이다. RLHF(Reinforcement Learning from Human Feedback, 인간 피드백 기반 강화학습)와 RLAIF(Reinforcement Learning from AI Feedback, AI 피드백 기반 강화학습)는 모델의 출력을 인간 또는 AI의 피드백으로 정렬하는 학습 방법이다. 해석가능성 연구(Mechanistic Interpretability)는 모델 내부에서 실제로 무엇이 일어나는지를 분석하는 분야다.

이것들이 논문에 머무르는 것이 아니다. Claude가 다른 모델보다 덜 환각하고 지시를 더 잘 따르는 이유 중 하나가 이 안전 연구의 직접적 반영이다. 안전과 성능이 트레이드오프가 아니라 같은 방향을 가리키는 셈이다.

5개 계층이 Claude에서 어떻게 매핑되는가

PART III에서 정리한 AI 활용의 5개 계층 구조가 Claude 생태계에서 구체적으로 구현된다.

계층	개념	Claude 구현
1. 프롬프트	사용자의 지시	Claude.ai 채팅, API messages
2. 컨텍스트 엔지니어링	정보 설계	Projects, Memory Tool, CLAUDE.md

3. 하네스	실행 환경	Claude Code (CLI, Desktop, Web, IDE)
4. MCP	외부 연결	MCP 서버 생태계
5. Skills	전문성 패키지	.claude/skills/ + 플러그인

이 표의 핵심은 각 행이 독립적 제품이 아니라 **하나의 진화 축 위에 놓여 있다**는 점이다. 1층 위에 2층이 쌓이고, 2층 위에 3층이 쌓인다. 프롬프트 없이는 컨텍스트 엔지니어링이 불가능하고, 컨텍스트 엔지니어링 없이는 하네스가 제대로 작동하지 않는다. 이 계층 구조를 이해하면 Claude 생태계 전체가 하나의 그림으로 보인다.

Claude Code가 코딩 도구에서 범용 에이전트로 진화한 이유

앞 글에서 Claude Code를 코드 에이전트로 소개했다. 여기서 한 걸음 더 들어간다. Claude Code는 코딩 도구가 아니라 범용 에이전트 인프라로 진화하고 있다.

2025년 9월, Claude Code SDK가 Claude Agent SDK로 이름이 바뀌었다. 이름 변경의 이유가 명확하다. 이 하네스가 코딩뿐 아니라 거의 모든 주요 에이전트 루프를 구동하게 됐기 때문이다. Anthropic 자체가 Claude Code를 **에이전트 하네스**라고 부른다.

PART III에서 하네스의 설계 철학을 다뤘다. **비즈니스 로직을 최소화하고 모델이 대부분의 일을 한다.** Claude Code는 이 철학의 대표적 구현이다. 하네스 자체는 가볍고, 모델의 능력에 의존한다. 이 설계가 범용성을 가능하게 한 것이다. 코딩에 특화된 로직이 아니라, 에이전트 루프라는 범용 구조 위에 서 있기 때문이다.

사용 가능한 형태는 네 가지다. CLI(터미널), 데스크톱 앱(Mac/Windows), 웹 앱(claude.ai/code), IDE 확장(VS Code, JetBrains). 형태는 달라도 하네스는 동일하다.

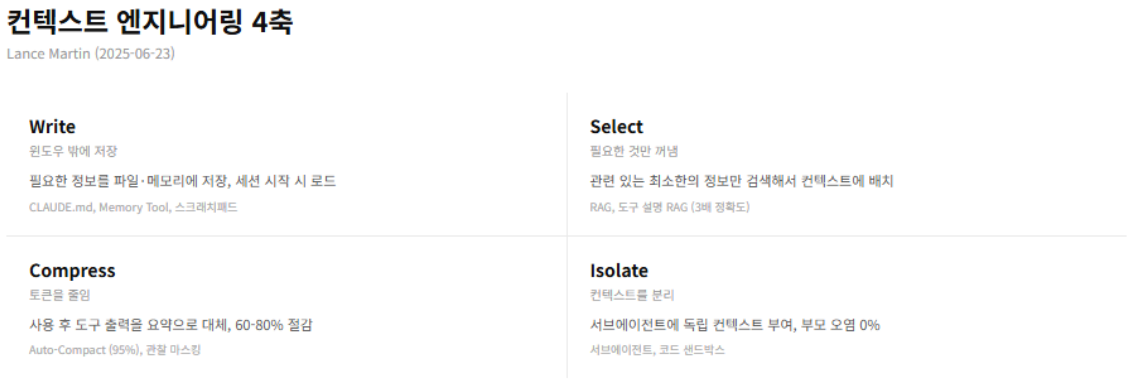

그림 4: Claude 생태계의 4축 구조. 프롬프트에서 Skills까지 각 계층이 어떻게 연결되는지를 보여준다.

이 그림에서 주목할 것은 계층 간의 화살표다. 아래층이 위층을 지탱하는 구조이며, 한 층이 무너지면 그 위의 모든 층이 영향을 받는다.

Claude 모델 계보: 2026년 4월 현재

Claude의 모델 라인업을 정리하면 다음과 같다.

모델	컨텍스트	특성
Opus 4.6	1M 토큰	최고 성능. 복잡한 추론에 적합
Sonnet 4.6	1M 토큰	Opus급 성능에 더 빠른 출력 속도
Sonnet 4.5	200K 토큰	성능과 속도의 균형
Haiku 4.5	200K 토큰	가장 빠르고 가장 저렴

Opus 4.6과 Sonnet 4.6이 1M 토큰 컨텍스트를 갖고 있다. PART III에서 **컨텍스트 윈도우가 크다고 다 채우면 안 된다**고 했지만, 넓은 공간이 있다는 것 자체가 컨텍스트 엔지니어링의 자유도를 높인다. 100K 토큰이 한계인 모델에서는 선택해야 했던 정보를, 1M 토큰 모델에서는 함께 넣을 수 있다.

Haiku 4.5는 비용에 민감한 대량 처리에 적합하다. 분류, 추출, 간단한 요약처럼 고난도 추론이 필요 없는 작업에서 Opus를 쓰는 것은 낭비다. 작업의 난이도에 맞는 모델을 선택하는 것이 비용 효율의 핵심이다. 앞 글에서 다룬 추론 인프라의 비용 구조를 모델 선택에 적용하는 셈이다.

MCP와 Skills가 만나는 지점

PART IV에서 MCP와 Skills를 각각 다뤘다. Claude 생태계에서는 이 둘이 함께 작동한다. 구체적 예를 보자.

이 이슈를 보고 PR을 만들어줘라고 요청하면, 세 계층이 동시에 작동한다. MCP가 GitHub에 접근해서 이슈를 가져오고 PR 생성 API를 호출한다. Skills가 **이 프로젝트에서 PR은 이런 형식이어야 하고, 리뷰어는 최소 2명이며, 테스트 커버리지는 80% 이상이어야 한다**는 규칙을 적용한다. 하네스가 이 전체를 중개하고, 컨텍스트를 관리하며, 검증 루프를 돌린다.

계층	역할	비유
MCP	외부 서비스 접근 (GitHub, Slack, DB)	손과 발
Skills	내부 전문성 (규칙, 형식, 품질 기준)	전문 지식
하네스	통합 관리 (컨텍스트, 도구 호출, 검증)	두뇌

이 세 계층이 Claude에서 가장 깊게 통합되어 있다. 이유는 단순하다. MCP와 Skills 둘 다 Anthropic이 설계했기 때문이다. 다른 플랫폼도 MCP를 도입했지만, Skills까지 통합한 것은 Claude 생태계가 유일하다.

> **MCP는 외부 세계에 대한 접근이고, Skills는 내부 전문성이다. 이 둘이 하네스 위에서 만날 때 에이전트는 단순 도구에서 전문가로 진화한다.**

Claude를 제대로 쓰는 다섯 가지 원칙

이 책에서 다뤄온 개념들을 Claude 생태계에서의 실천으로 압축하면 다섯 가지가 된다.

첫째, CLAUDE.md를 잘 쓴다. 프로젝트의 규칙, 구조, 금지 사항을 여기에 명시한다. 이 파일이 매 세션마다 자동 로드된다. 컨텍스트 엔지니어링의 Write 축을 실현하는 가장 직접적인 방법이다.

둘째, 세션을 짧게 끊는다. 컨텍스트 사용률이 70 80%를 넘으면 새 세션을 시작한다. PART III에서 다뤘듯이 컨텍스트가 가득 차면 모델의 주의력이 분산되고 품질이 떨어진다.

셋째, MCP는 최소한으로 유지한다. 필요한 외부 서비스만 연결하고, 서버당 도구 5개 이하를 유지한다. 도구가 너무 많으면 Context Confusion이 발생한다. 모델이 어떤 도구를 써야 할지 혼란스러워하는 현상이다.

넷째, Skills를 만든다. 프로젝트에 반복되는 절차가 있으면 스킬로 정의한다. PR 형식, 코드 리뷰 기준, 배포 절차 같은 것들이다. 한 번 정의하면 이후 모든 세션에서 일관되게 적용된다.

다섯째, 서브에이전트를 활용한다. 탐색 작업은 메인 컨텍스트에서 분리해서 서브에이전트에 맡긴다. 읽기 전용으로 격리하면 메인 컨텍스트가 오염되지 않는다. PART III의 Isolate 축을 실현하는 방법이다.

생태계 수준에서 본 Claude의 위치

개별 제품 비교를 넘어 생태계 수준에서 보면, Claude의 위치가 더 명확해진다.

계층	Claude	OpenAI	Google
모델	Opus/Sonnet/Haiku	GPT-4o/o3	Gemini 2.5
하네스	Claude Code (Agent SDK)	Codex, ChatGPT	Gemini Code Assist
프로토콜	MCP (개방형)	자체 도구 호출	자체 도구 호출
전문성 패키지	Skills	Custom GPTs	Gems

이 표에서 주목할 것은 프로토콜 행이다. MCP는 개방형 프로토콜이다. Anthropic이 만들었지만 Claude에만 묶여 있지 않다. OpenAI, Google 등 다른 플랫폼도 MCP를 도입할 수 있다. 실제로 2025 2026년에 걸쳐 여러 플랫폼이 MCP 지원을 추가하고 있다. 이것은 Anthropic의 전략적 선택이다. 프로토콜을 개방함으로써 생태계의 중심에 서려는 것이다.

USB가 특정 회사의 제품이 아니라 범용 표준이 된 것처럼, MCP가 AI 도구 연결의 범용 표준이 되면 그 표준을 설계한 Anthropic이 생태계의 중심에 위치하게 된다. 제품이 아니라 표준으로 경쟁하는 전략인 셈이다.

> **Claude 생태계의 차별점은 단일 제품의 성능이 아니라, 5개 계층이 일관된 설계 철학 아래 통합되어 있다는 점이다. MCP라는 개방형 프로토콜로 외부 생태계까지 연결하면서, 폐쇄와 개방 사이의 독특한 위치를 점유하고 있다.**

이것으로 PART VI을 마친다. 오픈소스 LLM, 추론 인프라, 코드 어시스턴트, Claude 생태계를 거치면서 AI가 실제로 작동하는 기반 구조를 살펴봤다. 기술의 원리를 아는 것과 그 기술이 돌아가는 인프라를 이해하는 것은 별개의 역량이다. 둘 다 갖춰야 AI를 제대로 쓸 수 있다.

Source: Anthropic — Claude 모델 계보, Claude Code, Constitutional AI; Thariq Shihipar 2025-09-29 — Claude Agent SDK 리네이밍; The Pragmatic Engineer 2025-09-23; MCP 사양 2024-11